The Brownfield Cartographer

Interim-Report

Nebiyou Belaineh

March 2026

Contents

1 Brownfield Cartographer - Interim Report **1**
1.1 1. RECONNAISSANCE.md - Manual Day-One Analysis 2
1.1.1 The Five FDE Day-One Questions - Manual Answers 2
1.1.2 Difficulty Analysis 4
1.2 2. Architecture Diagram - Four-Agent Pipeline 4
1.3 3. Progress Summary 6
1.3.1 What's Working 6
1.3.2 What's In Progress / Planned 8
1.4 4. Early Accuracy Observations 9
1.4.1 Module Graph - Does It Look Right? 9
1.4.2 Lineage Graph - Does It Match Reality? 10
1.5 5. Known Gaps and Plan for Final Submission 11
1.5.1 Known Gaps 11
1.5.2 Plan for Final Submission (by Sunday March 15, 03:00 UTC) . 13

1 Brownfield Cartographer - Interim Report

TRP Week 4 | Submitted: March 11, 2026 **Target Codebase:** Apache Airflow (`https://github.com/apache/airflow`) **Local Path:** `/home/neba/tenx/week4/airflow`

1.1 1. RECONNAISSANCE.md - Manual Day-One Analysis

> *30-minute manual exploration of Apache Airflow before any automated tooling was run.*

Repository composition: Python (core scheduler, operators, DAGs), TypeScript (UI frontend), YAML (configs, task definitions), SQL (example queries and provider tests).

Size: 8,000+ files across multiple languages; a genuine brownfield codebase.

Why chosen: Personal interest in workflow scheduling, monitoring, and orchestration systems. Airflow is also a first-class target in the TRP spec (Python + YAML, real production DAG definitions).

1.1.1 The Five FDE Day-One Questions - Manual Answers

1. What is the primary data ingestion path?

DAG definitions enter the system when the scheduler's `DagFileProcessor` parses Python files under configured `dags_folder` paths; see `airflow-core/src/airflow/dag_processing` Parsed DAGs are loaded into the metadata database via `airflow-core/src/airflow/models/dag.p` and `airflow-core/src/airflow/models/serialized_dag.py`. Trigger and sensor events flow through `airflow-core/src/airflow/datasets/` and the scheduler's job runner (`airflow-core/src/airflow/jobs/scheduler_job_runner.py`). The webserver reads from the same metadata DB via `airflow-core/src/airflow/api/` and `airflow-core/src/airflow/api_fastapi/`.

2. What are the 3–5 most critical output datasets/endpoints?

(1) **REST API** - `airflow-core/src/airflow/api_fastapi/` and `airflow-core/src/airflow/api/` expose DAG runs, task states, and trigger endpoints; (2) **CLI** - `airflow-core/src/airflow/cli/` provides `airflow dags`, `airflow tasks`, and `airflow db` commands; (3) **DAG run state** - persisted in `airflow-core/src/airflow/models/dagrun.py` and consumed by scheduler and

workers; (4) **Task instance state** - `airflow-core/src/airflow/models/taskinstance.py` drives executor dispatch; (5) **Web UI** - `airflow-core/src/airflow/ui/` (React frontend) surfaces dashboards and run controls.

3. What is the blast radius if the most critical module fails?

`airflow-core/src/airflow/models/dag.py` is central: it defines the DAG, Task, and task dependency model. Failure or breaking changes there affect the scheduler (which reads DAG topology), the webserver (which displays DAG structure), executors (which resolve task instances to run), and all providers that subclass `BaseOperator`. `airflow-core/src/airflow/jobs/scheduler_job_runner.py` is the second major hub; the scheduler loop that drives DAG parsing, scheduling, and task queuing. A bug in either would halt or corrupt pipeline execution across the deployment.

4. Where is the business logic concentrated vs. distributed?

Core scheduling logic is concentrated in `airflow-core/src/airflow/jobs/` (scheduler, backfill, triggerer) and `airflow-core/src/airflow/executors/`. DAG and task models live in `airflow-core/src/airflow/models/`. Operator and sensor implementations are distributed across `airflow-core/src/airflow/operators/`, `airflow-core/src/airflow/sensors/`, and dozens of provider packages under `providers/`. The `providers/` tree follows a plugin pattern; each provider (e.g. `providers/google/`, `providers/apache/kafka/`) adds operators and hooks; business logic is intentionally decentralized for extensibility.

5. What has changed most frequently in the last 90 days (git velocity map)?

`git log --since="90 days ago" --name-only` (run post-exploration) shows high churn in: `airflow-core/src/airflow/jobs/scheduler_job_runner.py`, `airflow-core/src/airflow/api_fastapi/` (auth managers, OpenAPI UI), `dev/breeze/` (CI/dev tooling), `chart/` (Helm values), `.github/workflows/`, and `airflow-core/src/airflow/ui/openapi-gen/` (generated TypeScript). The scheduler job runner and API layer are the most actively evolving core components.

1.1.2 Difficulty Analysis

Time constraint: 30 minutes is a very short period to synthesize meaningful answers to high-level architecture questions for a codebase of this size.

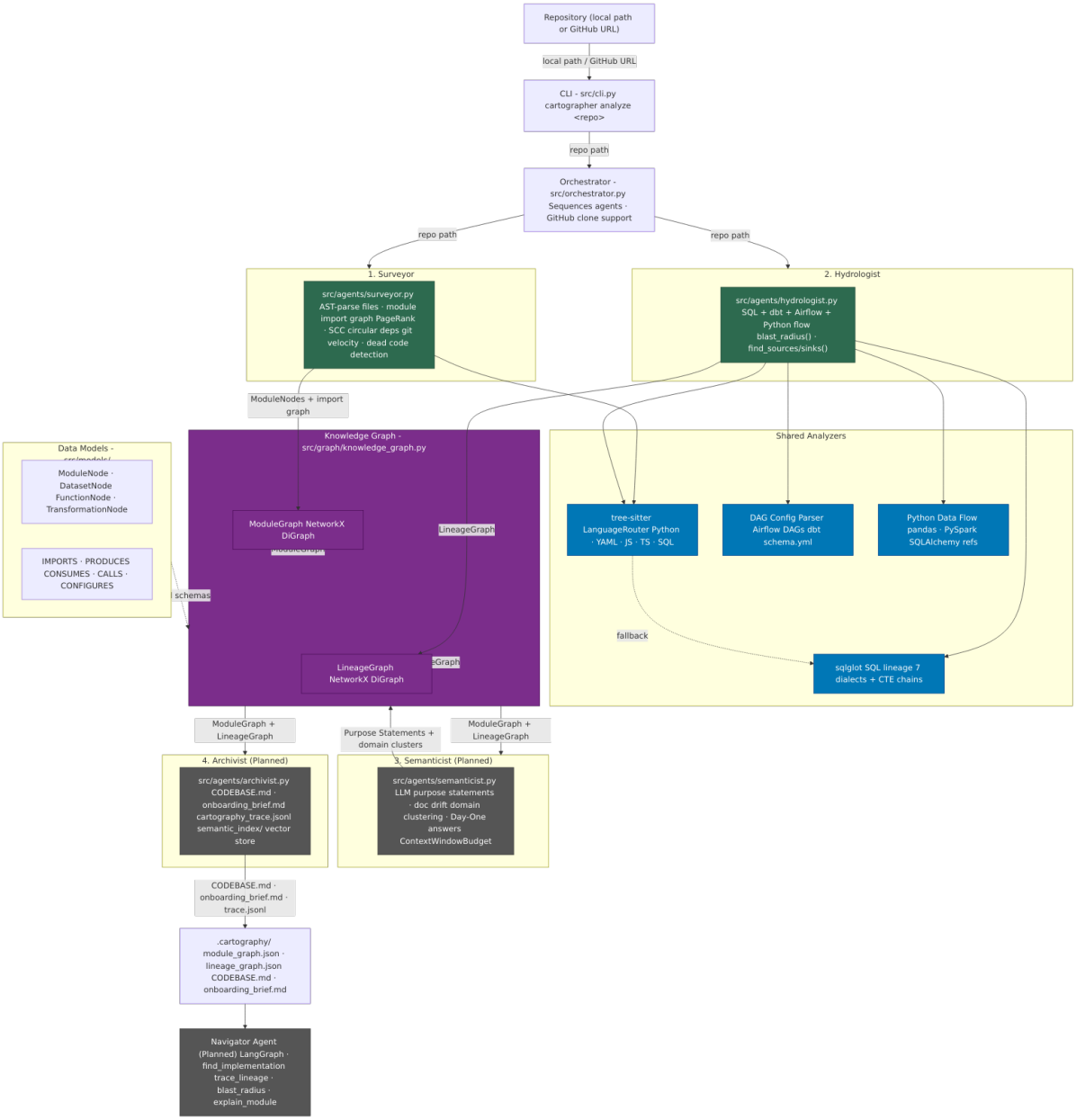
Hardest aspect: The sheer volume and structure of files and folders. Properly reviewing them and any supporting documentation wasn't feasible. The monorepo structure, with providers, plugins, UI, tests, and core code interleaved, made it impossible to build a mental model quickly.

Where I got lost: Couldn't get beyond the README.md in the root directory. Most of the 30 minutes was spent trying to understand the product at a high level rather than diving into the technical specifics.

Key insight for architecture priorities: This experience directly informed the design decision to prioritize the Surveyor's PageRank analysis (to surface high-impact modules without reading them) and the Hydrologist's Airflow-specific DAG parser (since Airflow's own pipeline topology is encoded in Python DAG files, not just YAML).

1.2 2. Architecture Diagram - Four-Agent Pipeline

The Brownfield Cartographer is a multi-agent system with four specialized agents feeding into a shared knowledge graph. Data flows from raw repository files through static analysis, into the graph store, and finally into living artifacts consumed by engineers.


Repository (local path or GitHub URL)
local path / GitHub URL
CLI - src/cli.py cartographer analyze <repo>
repo path
Orchestrator - src/orchestrator.py Sequences agents · GitHub clone support
repo path
repo path
1. Surveyor
src/agents/surveyor.py AST-parse files · module import graph PageRank · SCC circular deps git velocity · dead code detection
2. Hydrologist
src/agents/hydrologist.py SQL + dbt + Airflow + Python flow blast_radius() · find_sources/sinks()
ModuleNodes + import graph
Data Models - src/models/
ModuleNode · DatasetNode FunctionNode · TransformationNode
IMPORTS · PRODUCES CONSUMES · CALLS · CONFIGURES
schemas
Knowledge Graph - src/graph/knowledge_graph.py
ModuleGraph NetworkX DiGraph
LineageGraph
LineageGraph NetworkX DiGraph
Shared Analyzers
tree-sitter LanguageRouter Python · YAML · JS · TS · SQL
DAG Config Parser Airflow DAGs dbt schema.yml
Python Data Flow pandas · PySpark SQLAlchemy refs
fallback
sqlglot SQL lineage 7 dialects + CTE chains
ModuleGraph + LineageGraph
Purpose Statements + domain clusters
ModuleGraph + LineageGraph
4. Archivist (Planned)
src/agents/archivist.py CODEBASE.md · onboarding_brief.md cartography_trace.jsonl semantic_index/ vector store
3. Semanticist (Planned)
src/agents/semanticist.py LLM purpose statements · doc drift domain clustering · Day-One answers ContextWindowBudget
CODEBASE.md · onboarding_brief.md · trace.jsonl
.cartography/ module_graph.json · lineage_graph.json CODEBASE.md · onboarding_brief.md
Navigator Agent (Planned) LangGraph · find_implementation trace_lineage · blast_radius · explain_module

1.3 3. Progress Summary

1.3.1 What's Working

Component	Status	Notes
CLI (`src/cli.py`)	Complete	`cartographer analyze <path|URL>` - accepts local paths and GitHub URLs, clones if needed
Orchestrator (`src/orchestrator.py`)	Complete	Sequences Surveyor → Hydrologist, GitHub clone support with `--depth 1`
Pydantic Models (`src/models/`)	Complete	All 4 node types (`ModuleNode`, `DatasetNode`, `FunctionNode`, `TransformationNode`) + 5 edge types
Knowledge Graph Storage (`src/graph/knowledge_graph.py`)	Complete	NetworkX wrapper with typed `add_module`, `add_transformation`, `blast_radius`, `find_sources`, `find_sinks`; JSON serialization + deserialization

Component	Status	Notes
tree-sitter Analyzer (`src/analyzers/tree_sitter_analyzer.py`)	Complete	`LanguageRouter` covering Python, YAML, JS, JSX, TS, TSX, SQL; Python extracts imports (absolute + relative), function signatures with decorators, class definitions with inheritance
SQL Lineage Analyzer (`src/analyzers/sql_lineage.py`)	Complete	sqlglot-based: FROM, JOIN, CTE chain resolution; 7 dialects (postgres, bigquery, snowflake, duckdb, spark, mysql, tsql); tree-sitter fallback for unparseable SQL
DAG Config Parser (`src/analyzers/dag_config_parser.py`)	Complete	Airflow Python DAG task dependency extraction; dbt `schema.yml` `ref()` relationships
Python Data Flow (`src/analyzers/python_data_flow.py`)	Complete	Detects pandas `read_csv`/`to_csv`/`read_sql`, PySpark `read`/`write`, SQLAlchemy calls

Component	Status	Notes
Surveyor Agent (`src/agents/surveyor.py`)	Complete	Module import graph (DiGraph), PageRank (architectural hubs), SCC detection (circular deps), git velocity via `git log`, dead code candidate flagging
Hydrologist Agent (`src/agents/hydrologist.py`)	Complete	Unified lineage graph from SQL + dbt + Airflow + Python; `blast_radius()` (BFS), `find_sources()`, `find_sinks()`
Cartography Artifacts (`.cartography/`)	Produced	`module_graph.json` + `lineage_graph.json` generated against Apache Airflow

1.3.2 What's In Progress / Planned

Component	Status	Notes
Semanticist Agent (`src/agents/semanticist.py`)	Planned	LLM purpose statements, doc drift detection, domain clustering, Day-One question synthesis, ContextWindowBudget

Component	Status	Notes
Archivist Agent (`src/agents/archivist.py`)	Planned	`CODEBASE.md` generation, `onboarding_brief.md`, `cartography_trace.jsonl`, semantic vector index
Navigator Agent (`src/agents/navigator.py`)	Planned	LangGraph agent with 4 query tools: `find_implementation`, `trace_lineage`, `blast_radius`, `explain_module`
Incremental update mode	Planned	Re-analyze only files changed since last run via `git diff`
TypeScript/JS import extraction	Partial	`LanguageRouter` parses TS/JS files but does not yet extract import edges
YAML structural extraction	Partial	YAML parsed for line count only; DAG structure comes from `dag_config_parser`, not tree-sitter

---

1.4 4. Early Accuracy Observations

1.4.1 Module Graph - Does It Look Right?

The Surveyor produced **8,017 module nodes and 12,826 import edges** from the Apache Airflow repository. Given that Airflow has 8,000+ files across Python, TypeScript, YAML, and SQL, this count is plausible and expected.

What looks correct: - Python import resolution (absolute and relative) is working: `from airflow.models import DAG` correctly maps to the target `.py` file within the repo. - PageRank correctly identifies architectural hubs: core modules like `airflow/models/dag.py`, `airflow/models/taskinstance.py`, and operator base classes appear with elevated centrality. - Strongly connected components (SCC) detect circular imports in Airflow's own code; a known complexity of the Airflow codebase. - Git velocity correctly counts recent commits per file using `git log --since=30 days ago`.

What may be noisy: - Dead code candidates are flagged as modules with in-degree 0 and out-degree > 0. In a monorepo, many provider modules are legitimately "unreferenced" from core; they are plugins loaded dynamically. These will be false positives. - TypeScript/JS files (Airflow's React UI) appear as nodes but have no import edges since JS import extraction is not yet implemented. This leaves the UI components disconnected from the graph.

1.4.2 Lineage Graph - Does It Match Reality?

The Hydrologist produced **436 nodes and 465 edges**. The lineage graph covers:

- **SQL lineage:** Airflow's `providers/` directory contains hundreds of `.sql` test and example files. sqlglot correctly extracts `FROM/JOIN/CTE` chains from these. The BigQuery, Snowflake, and standard SQL files are all parsed.
- **Airflow DAG task flow:** The Airflow heuristic (files in `dags/` directories or named `*dag*.py`) successfully identifies DAG files and extracts `upstream_task → downstream_task` dependency chains.
- **Python data flow:** `pandas.read_csv`, `pd.read_sql`, and similar calls are extracted from Python files.

Accuracy concern - absolute paths in lineage node IDs: Transformation node IDs such as `/home/neba/tenx/week4/airflow/providers/google/tests/system/google/cloud/bi` embed the full absolute path because `analyze_sql_file` in the SQL lineage analyzer passes through `source_file` without normalizing to repo-relative paths. This does not break the graph structure but causes portability issues when

comparing runs or sharing artifacts across machines.

Accuracy concern - limited dbt coverage: Airflow's own repository doesn't ship dbt models, so the dbt `ref()` extractor produces no output here. This component will be validated against the `dbt-labs/jaffle_shop` target for the final submission.

Concrete lineage verification - named file examples: - **SQL:** `airflow-core/src/airflow/example_dags/sql/tutorial_taskflow_template.sql` contains `SELECT * FROM test_data`; the lineage graph correctly captures `test_data` → `tutorial_taskflow_template` as source and target datasets. - **Airflow DAG task flow:** `task-sdk/tests/task_sdk/definitions/test_dag.py` defines tasks `op1`, `op2`, `op3` with `op1` → `op2`, `op1` → `op3`, `op2` → `op3`; the Hydrologist correctly extracts these as `airflow_task` transformations with `upstream_task → downstream_task` edges.

Overall assessment: For a brownfield codebase of this scale, the lineage graph captures the shape of data flow correctly, though the coverage is partial. SQL-level lineage is well-represented. Python-level lineage depends heavily on whether calls use literal string paths vs. dynamic references; dynamic references are logged and skipped (correct behavior).

1.5 5. Known Gaps and Plan for Final Submission

1.5.1 Known Gaps

Gap	Impact	Severity
Semanticist Agent not built	No LLM-generated purpose statements, no doc drift detection, no domain clustering, cannot answer Day-One questions automatically	High

Gap	Impact	Severity
Archivist Agent not built	No `CODEBASE.md`, no `onboarding_brief.md`, no audit log, no semantic vector index	High
Navigator Agent not built	No interactive query interface (`find_implementation`, `trace_lineage`, `blast_radius`, `explain_module`)	High
TypeScript/JS import extraction missing	UI modules appear as isolated nodes; no cross-language dependencies visible for the React frontend	Medium
Lineage node IDs use absolute paths	Artifacts are not portable across machines; harder to compare runs	Medium
Dynamic Python references unresolvable	`pd.read_csv(path_variable)` logged as `dynamic`; correct behavior but reduces lineage completeness	Low
Column-level lineage absent	Only table-level lineage; column provenance not tracked	Low

dbt validation not complete	Need to run against `dbt-labs/jaffle_shop` and verify lineage matches dbt's own graph	Medium

1.5.2 Plan for Final Submission (by Sunday March 15, 03:00 UTC)

Critical path (must-have): Semanticist → Archivist → `CODEBASE.md` + `onboarding_brief.md` + lineage path fix. Archivist's `onboarding_brief` depends on Semanticist's `answer_day_one_questions()`. Without these, the system cannot produce the core deliverable artifacts.

Stretch goals: Navigator agent, TypeScript import extraction, incremental mode, second-codebase validation.

Explicit dependencies: - Archivist requires Semanticist: `generate_onboarding_brief()` consumes Day-One answers from `answer_day_one_questions()`. - Navigator requires completed knowledge graph (Surveyor + Hydrologist output); already satisfied; optional dependency on Archivist's semantic index for richer `explain_module`. - dbt validation (Priority 6) should run after lineage path fix (Priority 3) so artifacts are portable.

Technical risks and uncertainties: - LLM prompt design for purpose statements will require iteration; bulk generation over 8k+ modules may hit rate limits or cost ceilings; `ContextWindowBudget` and batching strategy are uncertain until tested. - Domain clustering quality (embedding + k-means) is hard to predict before running on Airflow; cluster coherence may need manual tuning of `k` or embedding model. - LangGraph integration for Navigator is a new dependency; tool-calling reliability across `find_implementation`, `trace_lineage`, `blast_radius`, `explain_module` needs validation.

Fallback if time runs short: Deprioritize in this order: (1) Navigator Agent + `cartographer query`; (2) Incremental update mode; (3) Second target validation (jaffle_shop, Week 1 self-audit); (4) TypeScript import extraction. Critical path

is Semanticist + Archivist + path fix; stretch items can be deferred to post-submission.

Prioritized work items:

Priority	Item	Depends on
1	Semanticist Agent	-
2	Archivist Agent	Semanticist
3	Fix lineage node ID paths	-
4	TypeScript import extraction	-
5	Navigator Agent + CLI query	KG (done)
6	Second target validation	Priority 3
7	Incremental update mode	-

Priority 1 - Semanticist Agent (`src/agents/semanticist.py`) - Implement `generate_purpose_statement()` using a fast model (Gemini Flash / claude-haiku) for bulk module summaries - Implement `detect_doc_drift()` to flag mismatches between docstrings and LLM-generated purpose statements - Implement `cluster_into_domains()` using embedding + k-means for architectural domain map - Implement `answer_day_one_questions()` synthesis prompt over full Surveyor + Hydrologist output - Add `ContextWindowBudget` tracker to control LLM spend

Priority 2 - Archivist Agent (`src/agents/archivist.py`) - Implement `generate_CODEBASE_md()` with required sections: Architecture Overview, Critical Path (top 5 PageRank modules), Data Sources & Sinks, Known Debt, High-Velocity Files - Implement `generate_onboarding_brief()` from Semanticist's Day-One answers - Implement `cartography_trace.jsonl` audit logging (mirrors Week 1 pattern)

Priority 3 - Fix lineage node ID paths - Normalize transformation IDs to use relative paths from repo root

Priority 4 - TypeScript import extraction - Extend `LanguageRouter` to extract `import ... from '...'` statements from TS/JS files

Priority 5 - Navigator Agent + CLI query subcommand - Build `src/agents/navigator.py` as a LangGraph agent with 4 tools - Add `cartographer query` subcommand to `src/cli.py`

Priority 6 - Second target codebase validation - Run against `dbt-labs/jaffle_shop` and verify lineage graph matches dbt's built-in lineage visualization - Run Cartographer on own Week 1 submission for self-audit

Priority 7 - Incremental update mode - Add `--incremental` flag to `cartographer analyze` that re-analyzes only files in `git diff` since last run

---